Enhancement Three Narrative

Reece Martin

1. Artifact Description

   The artifact I selected is the Travlr Getaways full-stack web application that I originally created in CS 465. The project includes a customer-facing travel website, an Angular administrative application, a Node.js and Express backend, REST API, and a MongoDB database. The database contains information like trip codes, names, dates, resorts, prices, images, and descriptions. For this enhancement, I focused on improving how trip data is stored, validated, and retrieved from MongoDB.

2. Justification

   I selected this artifact because it demonstrates my ability to work with a database in a full-stack application. The original project already used a Mongoose schema with required fields and indexes, so I focused on making improvements past that. I added stronger validation by trimming unnecessary spaces and preventing empty values. I also made the trip code unique. The original application did not prevent duplicate codes, so I added a unique index in MongoDB. Duplicate trip codes are now rejected.

   I also added pagination to the trip-list query. Instead of requiring the API to return the entire trip collection every time, the API can now return smaller groups of records. Originally, the backend would crash if a duplicate trip code was added, so I improved error handling to return an error rather than stopping the server.

3. Course Outcomes

   This enhancement supports course outcome 4 because I used MongoDB, Mongoose, JavaScript, and Express to improve the way the application manages and retrieves its data. The changes make the database portion of the application more reliable and better prepared to handle a larger collection of trips. Pagination also demonstrates how database queries can be designed so that the application does not always need to retrieve every record at once.

4. Reflection

   Completing this enhancement helped me understand that database design involves more than simply storing and retrieving information. One challenge I encountered was discovering that adding “unique: true” to the Mongoose schema did not automatically change the existing MongoDB index into a unique index. I had to remove the existing trip-code index and recreate it as a unique index after deleting the duplicates I made for testing. I also discovered that invalid data could cause the backend to stop running, so I added error handling to the add and update functions.